



Université de Namur
Faculté d'Informatique

IEDC001 – Projet individuel

CondruLytics

Analyse, nettoyage et valorisation des résultats
du Challenge Condrusien
de 2006 à 2025

Etudiant : Mestré Michaël
Professeur : Frénay Benoît
Année académique : 2025-2026

26 mai 2026

Table des matières

1	Introduction	1
2	Contexte du projet	1
3	Objectifs	2
4	Extraction des données et préparation	2
4.1	Architecture générale	2
4.2	Extraction des fichiers ZIP	3
4.3	Parsing des fichiers PDF	3
4.4	Consolidation	4
4.5	Nettoyage et harmonisation	4
4.6	Modèle de données	5
4.7	Service PostgreSQL avec Docker	6
5	Feature engineering	6
5.1	Variables individuelles	7
5.2	Variables de forme récente	7
5.3	Tendance de performance	7
5.4	Segmentation par distance	7
5.5	Normalisation par course	7
6	Modélisation prédictive	7
7	Visualisation interactive	7
8	Technologies utilisees	8
9	Limites identifiees	8
10	Perspectives d’amelioration	8
11	Conclusion	9

1 Introduction

Le projet *CondruLytics* a pour objectif de transformer des résultats historiques des courses du **Challenge Condrusien** en un jeu de données exploitable pour l'analyse, la visualisation et la modélisation. Les données de départ sont principalement des classements au format PDF, publiés sur plusieurs années et selon des formats variables. Cette hétérogénéité rend nécessaire la mise en place d'un pipeline capable d'extraire les informations, de les harmoniser et de les stocker dans une base de données qui représentera la source stable pour la suite des analyses.

Le projet couvre les principales étapes d'un pipeline de données : extraction des fichiers, parsing des PDF, consolidation, nettoyage, chargement dans une base PostgreSQL, création de variables, apprentissage automatique pour prédire le temps des coureurs sur différentes courses et enfin visualisation interactive via le framework Python Streamlit ¹.

2 Contexte du projet

Le **Challenge Condrusien** ² est un challenge de course à pied amateur sur routes et chemins vallonnés qui regroupe une vingtaine d'organisation chaque année dans le Condroz liégeois. Chaque organisation propose différentes distances : une petite sous les 7km, une longue entre 10 et 17km et parfois une version trail de plus de 20km. Les classements contiennent notamment le nom des coureurs, leur position générale, leur catégorie, leur temps, leur club, la distance de la course et la date de l'épreuve. Les fichiers ZIP de départ proviennent de la page officielle des archives du Challenge Condrusien ³. Les données d'archives couvrent la période 2006-2025 à l'exception de l'année 2020 en raison de la pandémie de COVID-19.

La difficulté principale provient du fait que les classements ne suivent pas un format unique. Les PDF présentent des structures différentes : colonnes absentes, ordre variable des champs, présence ou non du numéro de dossard, informations de course encodées dans le texte, ou encore changements de mise en page selon les années. Le projet répond à cette contrainte par l'utilisation de plusieurs parseurs spécialisés. Après cette étape d'extraction, les données sont nettoyées pour harmoniser les catégories, normaliser les temps et corriger les erreurs d'extraction. Le format des noms des coureurs sont standardisés et certaines erreurs de saisie sont corrigées. La base de données relationnelle créée permet ensuite de structurer les données pour des analyses, des expérimentations de modélisation et différentes visualisations interactives.

1. <https://streamlit.io>

2. <https://challengecondrusien.com>

3. <https://challengecondrusien.com/archives/>

3 Objectifs

Les objectifs du projet peuvent être regroupés selon les grands axes suivants :

1. **Construire un jeu de données fiable** à partir de fichiers PDF historiques.
2. **Harmoniser les informations** relatives aux coureurs, aux courses, aux catégories et aux performances.
3. **Mettre en place une base de données stable** permettant de consulter les résultats, les coureurs et les courses.
4. **Préparer des variables de modélisation** afin d'analyser l'évolution des performances et d'expérimenter des approches prédictives.
5. **Créer des visualisations interactives** pour explorer les données et communiquer les résultats.

4 Extraction des données et préparation

4.1 Architecture générale

Le projet est organisé sous forme de modules Python séparés par responsabilité. Cette décomposition facilite la maintenance et rend le pipeline reproductible.

Module	Role principal
<code>src/extraction</code>	Décompression des archives ZIP contenant les classements bruts.
<code>src/parsing</code>	Extraction des données depuis les fichiers PDF.
<code>src/cleaning</code>	Nettoyage, normalisation et correction des données extraites.
<code>src/loading</code>	Définition du schema relationnel et chargement vers PostgreSQL.
<code>src/utils</code>	Centralisation des fonctions utilitaires globales.
<code>src/feature_engineering</code>	Création de variables dérivées pour l'analyse avancée et la modélisation.
<code>src/streamlit_app</code>	Interface interactive de consultation des données.
<code>notebooks</code>	Analyses exploratoires, régression et clustering.

Un système de logging est mis en place pour suivre l'exécution du pipeline et identifier les éventuelles erreurs ou anomalies.

Le point d'entrée principal est le fichier `src/pipeline.py`. Celui-ci permet d'exécuter l'ensemble du pipeline ETL ou une étape spécifique via l'argument `-step`. Les étapes disponibles sont : `unzip`, `parse`, `build`, `clean` et `load`.

Les données intermédiaires sont stockées au format Parquet dans `data/processed/parquet`, ce qui permet de conserver une trace des résultats de chaque étape et de faciliter les reprises en cas d'erreur.

Les autres modules sont exécutés de manière séparée pour des tâches spécifiques, notamment la création de variables, ce qui rend le projet plus flexible et modulaire.

4.2 Extraction des fichiers ZIP

La première étape consiste à décompresser les archives ZIP situées dans `data/raw/zip`. Ces archives ont été récupérées depuis la page officielle des archives du Challenge Condrusien⁴, qui centralise les classements historiques disponibles. Le module `extraction.unzip` parcourt récursivement les archives, extrait les fichiers et produit un rapport de synthèse. Ce rapport indique le nombre de fichiers extraits, le nombre de fichiers ignorés car déjà présents et le volume total traité.

4.3 Parsing des fichiers PDF

Le parsing est l'étape la plus sensible du projet. Les fichiers PDF sont convertis en tableaux de données grâce à la bibliothèque `pdfplumber`. Le projet utilise un patron de conception de type `Strategy` et un `ParserFactory`, choisit automatiquement le parseur adapté au fichier analyse.

Les stratégies implémentées permettent d'extraire toutes les données sur la plage de temps analysé et sont les suivantes :

- `Parser2006_2007`, adapté aux classements des années 2006 et 2007 ;
- `Parser2008_2016`, adapté aux formats intermédiaires sans URL dans le texte ;
- `Parser2017_2024`, adapté aux formats plus récents contenant des marqueurs web ;
- `Parser2025`, adapté aux classements produits par le chronometreur `Goal Timing`.

Chaque parseur extrait les champs principaux suivants : position, nom du coureur, club, sexe, catégorie, position dans la catégorie, temps, nom de la course, date et distance. Un identifiant de course, appelé `racekey`, est ensuite construit à partir du nom de la course, de la date et de la distance.

Le service de parsing traite les fichiers en parallèle avec `ThreadPoolExecutor`. Il génère également un rapport indiquant le statut de chaque fichier : succès, fichier ignoré, erreur ou différence entre le nombre de participants attendu et le nombre de lignes parsées.

Pour chaque fichier traité, un fichier Parquet est produit dans `data/processed/parquet` avec les données extraites. Ces fichiers intermédiaires sont ensuite utilisés pour les étapes suivantes du pipeline.

4. <https://challengecondrusien.com/archives/>

4.4 Consolidation

Après le parsing, les fichiers Parquet intermédiaires sont consolidés dans un dataset unique. Cette étape concatène les résultats disponibles dans `data/processed/parquet`, supprime les éventuels doublons sur base de `racekey` et du nom du coureur, puis écrit le fichier `data/processed/results.parquet`.

4.5 Nettoyage et harmonisation

Le nettoyage est effectué par le module `cleaning.clean`. Il vise à rendre les données homogènes malgré les variations historiques de format. Les principales opérations sont :

- harmonisation des catégories sportives vers des libellés standards ;
- conversion des positions en entiers ;
- normalisation des temps au format `HH:MM:SS` puis conversion en secondes ;
- correction manuelle de certaines courses dont le nom ou la distance est mal extrait ;
- suppression de noms de coureurs invalides ou non exploitables ;
- correction ponctuelle de données coureurs connues ;
- normalisation des noms des coureurs afin de réduire les doublons ou erreurs orthographiques.

Une partie complexe du nettoyage concerne la correction du nom des coureurs. Dans les classements historiques, une même personne peut apparaître sous plusieurs formes : inversion du nom et du prénom, accentuation différente, caractères parasites, espaces multiples, fautes de frappe ou variations dues à l'extraction PDF. Se limiter au nom brut risquerait donc de considérer une même personne comme plusieurs individus distincts.

Le module `cleaning.clean_names` applique d'abord une normalisation des chaînes de caractères : passage en majuscules, suppression des caractères non alphabétiques (tirets, apostrophes) et réduction des espaces multiples. Cette opération crée une colonne `Nom_clean`, utilisée comme base de comparaison.

La correction ne repose cependant pas uniquement sur la similarité du nom. Le code construit un profil par coureur à partir de plusieurs informations :

- le nombre d'apparitions du nom dans les résultats ;
- le sexe majoritaire observé ;
- la vitesse moyenne du coureur ;
- l'ensemble des courses auxquelles le coureur est associé, pour les coureurs fréquents.

Cette logique permet d'éviter des corrections trop agressives. Deux noms proches ne sont pas automatiquement fusionnés : ils doivent aussi être compatibles avec le profil sportif observé. Par exemple, deux profils de sexe différent ne sont pas rapprochés. De même,

deux coureurs fréquents présents dans une même course ne peuvent pas être considérés comme une seule et même personne, car une personne ne peut pas avoir deux résultats distincts dans la même épreuve.

La méthode fonctionne en deux passes.

1. **Première passe : coureurs fréquents entre eux.** Les coureurs apparaissant au moins trois fois sont comparés deux à deux. La similarité du nom est calculée avec un score de fuzzy matching à l'aide de la bibliothèque **RapidFuzz**. Ce score est combiné avec un score de proximité de vitesse. Le score final donne un poids dominant au nom, mais conserve une vérification par la performance. Si deux profils sont suffisamment proches, le nom le plus fréquent devient le nom corrigé. Cette première passe sur les coureurs fréquents permet de créer une base de noms fiables pour la suite du processus.
2. **Deuxième passe : coureurs rares vs coureurs fréquents.** Après la première correction, les noms apparaissant moins de trois fois sont comparés aux coureurs fréquents. Pour limiter les comparaisons inutiles, une clé de blocage est créée à partir des premières lettres des parties du nom. Le fuzzy matching est ensuite appliqué uniquement à des candidats plausibles. Une correction est acceptée seulement si le score du nom et le score final dépassent des seuils élevés.

Cette approche est volontairement prudente. L'objectif n'est pas de corriger tous les cas possibles, mais uniquement les cas pour lesquels le niveau de confiance est élevé. Cela signifie que certains doublons peuvent subsister, notamment lorsque le nom est très déformé, lorsque le coureur apparaît peu de fois ou lorsque son profil de coureur est insuffisant. En contrepartie, le risque de fusionner deux personnes différentes est réduit. Ce compromis est important pour ce travail d'analyse : il est préférable de conserver quelques doublons résiduels plutôt que de fusionner accidentellement des individus distincts.

La fin de l'étape de nettoyage produit `data/cleaned/results.parquet`. Elle est essentielle car les analyses et les modèles dépendent fortement de la qualité des variables temporelles, des catégories et des noms individuels de coureurs.

4.6 Modèle de données

Les données nettoyées sont chargées dans une base de données PostgreSQL selon un schema en étoile simple compte tenu de la nature plate des données et des analyses prévues.

Table	Description
<code>dim_runner</code>	Dimension des coureurs. Elle contient l'identifiant du coureur, le nom original, le nom nettoyé et le sexe.

<code>dim_race</code>	Dimension des courses. Elle contient l'identifiant de course, la clé <code>racekey</code> , le nom, la distance, la date et l'année.
<code>fact_results</code>	Table de faits contenant une ligne par résultat : coureur, course, position, position categorie, temps, vitesse, catégorie et fichier source.

Bien que la catégorie puisse être considérée comme étant un attribut du coureur, elle est stockée dans la table de faits car elle peut varier d'une course à l'autre pour un même coureur (par exemple, un coureur peut être dans la catégorie "Senior" puis basculer dans la catégorie "Vétéran1" ensuite). De plus, cela permet de conserver une trace de la catégorie telle qu'elle a été observée dans les résultats, même si elle est parfois incohérente ou mal extraite.

Le chargement utilise des caches en mémoire pour un accès rapide et pour éviter de recréer plusieurs fois les mêmes coureurs ou les mêmes courses. Les résultats sont insérés par lots afin d'améliorer les performances d'écriture.

4.7 Service PostgreSQL avec Docker

Pour ce développement, la base de données PostgreSQL est fournie par un service Docker défini dans `docker/docker-compose.yml`. Ce fichier décrit un conteneur PostgreSQL nommé `condrulytics_postgres`, expose le port 5432 et crée une base `condrulytics` avec un utilisateur dédié. Un volume Docker, `postgres_data`, est associé au service afin de conserver les données entre deux redémarrages du conteneur. Cette approche permet d'utiliser rapidement une base de données locale, sans dépendre d'une installation PostgreSQL propre à une machine. Le code Python communique avec la base via SQLAlchemy et une configuration de connexion, ce qui signifie que la base Docker peut très facilement être remplacée par une base distante.

Cette séparation entre le pipeline, l'application et le service de base de données donne au projet une structure modulaire et évolutive : les composants sont isolés, remplaçables et plus faciles à déployer progressivement.

5 Feature engineering

Le module `feature_engineering.py` prépare un jeu de données destiné aux analyses avancées et à la modélisation. Les données sont lues depuis PostgreSQL puis enrichies avec plusieurs variables.

5.1 Variables individuelles

Pour chaque coureur, le code calcule le nombre de courses, la vitesse moyenne, l'écart-type des vitesses, la meilleure vitesse, la moins bonne vitesse, la première et la dernière année de participation, la durée de carrière et le nombre moyen de courses par an.

5.2 Variables de forme récente

La forme récente d'un coureur est approximée par des moyennes glissantes sur ses performances précédentes. Deux variables sont créées : une moyenne sur trois courses et une moyenne sur cinq courses. Le décalage d'une ligne avant calcul permet d'éviter d'utiliser la performance courante pour prédire ou expliquer cette même performance ce qui introduirait une fuite de données (*data leakage*).

5.3 Tendance de performance

Une tendance linéaire de vitesse est estimée pour chaque coureur à l'aide d'une régression polynomiale simple de degré 1. Cette pente donne une indication de progression ou de régression globale dans le temps.

5.4 Segmentation par distance

Les distances sont réparties en trois groupes : courtes, moyennes et longues. Les seuils sont définis par les quantiles 33% et 66%, ce qui rend la segmentation adaptée à la distribution réelle des courses observées.

5.5 Normalisation par course

Pour comparer les performances entre courses, le code calcule la moyenne et l'écart-type de vitesse au sein de chaque course. Une variable `speed_z_race` indique ensuite la position relative d'un coureur par rapport aux autres participants de la même épreuve.

6 Modélisation prédictive

7 Visualisation interactive

Le projet inclut une application Streamlit nommée *CondruLytics Dashboard*. Elle interroge la base PostgreSQL et propose plusieurs vues :

- une vue d'ensemble des coureurs les plus présents et des courses les plus fréquentées ;
- une analyse par course, avec les résultats classés par position ;

- une analyse par coureur, permettant de suivre les temps et performances au fil des années.

Cette interface rend les données accessibles sans devoir manipuler directement des scripts Python ou des requêtes SQL.

8 Technologies utilisées

Le projet repose principalement sur l'écosystème Python. Les bibliothèques centrales sont **pandas** et **numpy** pour la manipulation de données, **pdfplumber** pour l'extraction depuis les PDF, **SQLAlchemy** et **psycopg2** pour l'interaction avec PostgreSQL, **scikit-learn** pour les expérimentations de modélisation, et **Streamlit** pour la visualisation interactive.

Un fichier `docker-compose.yml` fournit un service PostgreSQL local, ce qui facilite la reproductibilité de l'environnement de stockage.

9 Limites identifiées

Malgré la structuration du pipeline, plusieurs limites doivent être prises en compte.

- Les PDF historiques restent hétérogènes, ce qui impose des corrections spécifiques et peut entraîner des erreurs de parsing.
- Certaines corrections sont actuellement encodées manuellement, par exemple pour des noms de courses ou des distances particulières.
- L'identification des coureurs repose principalement sur le nom nettoyé ; des homonymes ou des changements d'orthographe peuvent donc affecter les analyses longitudinales.
- Certaines pages Streamlit construisent des requêtes SQL à partir de valeurs sélectionnées, ce qui pourrait être sécurisé davantage par des requêtes paramétrées.
- La validation statistique des modèles prédictifs n'est pas encore formalisée dans le pipeline principal.

10 Perspectives d'amélioration

Plusieurs améliorations peuvent être envisagées :

- ajouter des tests automatisés pour les parseurs et les fonctions de nettoyage ;
- externaliser les corrections manuelles dans des fichiers de configuration ;
- enrichir l'identification des coureurs avec des méthodes de rapprochement flou ;
- documenter les indicateurs de qualité produits par les rapports de parsing ;

- integrer les notebooks de regression et de clustering dans une demarche experimentale reproductible ;
- completer le dashboard avec des filtres par annee, distance, sexe et categorie.

11 Conclusion

Le projet *CondruLytics* met en place une chaine de traitement complete pour transformer des classements PDF historiques en une base de donnees exploitable. L'approche modulaire permet de traiter des formats de fichiers differents, de nettoyer les donnees, de les stocker dans un schema relationnel et de les valoriser via des analyses et une application interactive.

D'un point de vue academique, le projet illustre une demarche de data engineering appliquee a des donnees sportives reelles : collecte, extraction, structuration, nettoyage, modelisation des entites, creation de variables et restitution visuelle. Les prochaines etapes consisteraient a renforcer la validation des donnees et a formaliser davantage l'evaluation des modeles predictifs.